The lux Hub Replicator: a Z Specification

Formal model of the single-writer, two-lock replication discipline

July 2026

1 Introduction

An MCP `clear` call once froze an agent for about 38 minutes: the tool sent to the display synchronously, on the agent's own thread, over a socket with no send timeout, while holding the client send lock. The fix moves every send to the display onto one background worker inside the Hub — the *replicator* — and keeps every agent-facing tool on the Hub's in-memory store, which it updates and returns from at once.

The worker introduces real concurrency. It is a background thread; it drains a set of changed scenes that several other threads mark; it reads a store that those threads mutate; and it kills and restarts a wedged display while those threads keep running. This document models that concurrency as a finite state machine over a small fixed set of scenes and mutator threads, so that ProB can enumerate every interleaving and either confirm the invariants or produce the exact offending trace.

Six properties must hold across every interleaving:

I1 — single writer. Only the worker ever writes to the display connection. No mutator thread holds the client send lock.

I2 — agent-path liveness. No mutator operation is ever disabled by display I/O or by the client send lock. A mutator makes progress even while the worker is stuck sending to a dead display.

I3 — no lock cycle. No thread ever holds the store lock and the client send lock at the same time. The worker snapshots under the store read lock, releases it, and only then takes the client lock to send.

I4 — no lost update. Whenever the system comes to rest with the display up, the display shows the store's latest state for every scene — including after a reap and respawn, which re-marks every live scene, and including after a cycle that failed and restored its own batch.

I5 — one display. The worker never spawns a fresh display while one is still alive; a respawn follows a kill.

I6 — menu-delivery durability. A menu change that has been marked is either delivered to the display by the worker's own send or stays marked; a failed cycle never drops it. When the system comes to rest, no marked menu change is left undelivered.

Recovery and blanking. The worker's cycle can fail in three ways, and an emptied scene must disappear from the display; both shape the model. A send raises one of two socket errors — a timeout (`BlockingIOError`) or a dead peer (`OSError`) — and each has its own heal. But the heal itself can fail: an unspawnable display or a refused reconnect raises before the work is re-marked, and any other error inside the cycle escapes the two socket handlers. That third, generic failure must not drop the drained work: the worker re-marks it — every scene the cycle drained *and* the clear it carried *and* the menu flag it carried — and backs off. The two socket heals re-mark more: the whole drained batch, together with every scene the store still holds live, together with the carried clear and the carried menu flag. The drained scenes matter because a batch can drain a scene whose only job is to be blanked, and a reconnect can land on the same still-running display that keeps its stale copy, so a lost blank must be re-queued or the departed scene lingers forever; blank-then-repaint is idempotent. Finally, a drained scene whose store copy is empty is pushed as a blank into its own recorded frame, so an emptied or departed scene vanishes — unless the batch already carried a clear, which blanked the whole display, in which case the empty scene is skipped rather than blanked twice.

The menu leg. Alongside the scenes and the clear, the worker also pushes the agent menu bar and the registered World-menu items. A menu change is a payload-less flag, like the clear: a mutator sets it, and the worker reads the menu registry *fresh* at send time, pushing whatever the registry holds then. Reading fresh is the whole point of the payload-less design — a menu change that lands while an earlier send is failing is never overwritten by a stale re-mark, because there is no payload to go stale; the newest registry state always wins. The menu push is one send with the same time limit and the same two socket failures as a scene send, so a wedged or dead display on the menu push heals exactly as it does on a scene push, and the carried menu flag is re-marked so the healed display is repainted.

2 Basic Types

$[SCENE, MUTATOR]$

`SCENE` is the set of scenes the Hub can hold; two suffice to exhibit a coalesced batch and a per-scene lost update. `MUTATOR` is the set of threads that change the store: MCP tool calls (`show`, `update`, `clear`) and Hub-side click handlers. Both kinds do the same thing to the store, so one carrier covers both; two mutators exhibit lock contention. Both carriers are bounded by ProB's `DEFAULT_SETSIZE`.

3 Free Types

$VAL ::= vempty \mid vlive \mid vtorn$

The content of one scene in the store or on the display, abstracted to a token. `vempty`: no elements (a scene that has been cleared or never populated). `vlive`: a committed, consistent scene tree. `vtorn`: the half-updated state a scene passes through while a mutator is inside a multi-step `replace_scene` — the value that, if read without the lock, is the "dictionary changed size during iteration" defect. Real scene structure is elided; what matters is whether a read observes a committed value or a torn one.

$DPROC ::= dup \mid dstuck \mid ddead$

The display process as the worker's send sees it. `dup`: alive and draining its socket; a send succeeds. `dstuck`: alive but no longer reading, so the send buffer fills and the time-limited send raises `BlockingIOError`. `ddead`: the peer is gone, so the send raises a plain `OSError` (`ECONNRESET`/`EPIPE`).

$LOCK ::= held \mid free$

$FLAG ::= set \mid clear$

Two-valued markers for the worker's read lock and client lock, and for the cleared, shutting, and menu signals, respectively. A dedicated free type is used rather than a built-in boolean so the spec is self-contained for `fuzz`.

$$
\begin{aligned}
&WPHASE ::= \\
&\quad widle \mid wdrain \mid wblank \mid wmenu \mid wproc \mid wsnap \mid wsend \mid wmsend \mid \\
&\quad wreap \mid wensure \mid wremark \mid wreconn \mid \\
&\quad wflush \mid wstopped
\end{aligned}
$$

The worker's own program counter. `widle`: waiting on the condition. `wdrain`: woken, about to take the whole changed-scene set. `wblank`: batch drained, about to blank the display if the batch carried a clear. `wmenu`: display blanked (or not), about to push the menu registry if the batch carried a menu change. `wproc`: iterating the drained batch. `wsnap`: holding the store read lock, copying one scene's roots. `wsend`: holding the client lock, sending that copy. `wmsend`: holding the client lock, sending the menu push. `wreap`/`wensure`/`wremark`: the reap-respawn-remark recovery after a send timeout. `wreconn`: the reconnect recovery after a dead peer. `wflush`/`wstopped`: the final bounded flush at shutdown. A generic cycle failure returns the worker straight to `widle` after restoring its batch.

4 State

The state is flat: one schema. Its predicate carries only *structural* coherence — the locks and the snapshot hold at most one thing, and the write lock is held exactly when a scene is mid-replace. These hold in every design, correct or buggy, so they constrain the state space without masking the safety properties. The safety properties (Section 10) are deliberately *absent* here, so that a violating state stays reachable and ProB can find it.

Repl

$$
\begin{array}{l}
store : SCENE \longrightarrow VAL \\
displayed : SCENE \longrightarrow VAL \\
dirty : \mathbb{P}\ SCENE \\
batch : \mathbb{P}\ SCENE \\
drained : \mathbb{P}\ SCENE \\
cleared : FLAG \\
batchCleared : FLAG \\
menusDirty : FLAG \\
batchMenus : FLAG \\
menuShown : FLAG \\
snapScene : \mathbb{P}\ SCENE \\
snapVal : VAL \\
storeLockMut : \mathbb{P}\ MUTATOR \\
mutScene : \mathbb{P}\ SCENE \\
readLock : LOCK \\
clientLock : LOCK \\
display : DPROC \\
shutting : FLAG \\
wphase : WPHASE \\
\hline
\#storeLockMut \leq 1 \\
\#snapScene \leq 1 \\
\#mutScene \leq 1 \\
(storeLockMut = \varnothing \Leftrightarrow mutScene = \varnothing)
\end{array}
$$

`store` is the Hub's authoritative copy; `displayed` is what the display is currently showing. `dirty` is the changed-scene set guarded by the worker's condition, and `cleared` the cleared flag beside it — both live signal state that a mutator sets and the worker resets when it drains. `batch` is the set of scenes still to send this cycle — it shrinks as each is pushed — while `drained` is the whole set the cycle drained, kept unshrunk so a generic failure can restore all of it. The two mirror the shipped `DrainedBatch`: its `scenes` field is immutable (`drained`), and the worker's loop index over it is what `batch` tracks. The distinction is load-bearing: a generic failure of a batch that carried a clear must re-mark *every* scene it drained, not just the unsent remainder, because the re-marked clear will blank the whole display and every drained scene must then be repainted. `batchCleared` is the clear that *that* drained batch carried: the worker snapshots the cleared flag into `batchCleared` when it drains, resets the live flag, and keeps `batchCleared` for the whole cycle, so both the empty-scene skip and a recovery re-mark can still see that the batch was a clearing one. Modelling the two separately is what makes the recovery re-mark of a consumed clear expressible; a single flag, reset when the blank is pushed, would have forgotten by the time a send failed.

`menusDirty` is the live menu-change flag beside `cleared`: a mutator sets it when the menu registry changes, and the worker resets it when it drains. `batchMenus` is the menu flag *that* drained batch carried — snapshotted from `menusDirty` at the drain and kept for the whole cycle, exactly as `batchCleared` carries the clear — so a recovery re-mark can see that the failed batch owed a menu push. `menuShown` records whether the worker has delivered the current menu registry to the display since the last menu change: `set` once a menu push has succeeded, `clear` once a change is marked and not yet delivered. It is a statement about the worker's *delivery obligation*, not about a menu payload, because the menu is payload-less and read fresh at send time; it is the variable I6 is stated over.

`snapScene`/`snapVal` are the one scene the worker has copied out under the read lock and the value it copied. `storeLockMut` names the mutator holding the store write lock (empty or one), and `mutScene` the scene it is mid-replacing. `readLock` and `clientLock` record whether the worker holds the store read

lock and the client send lock. `display` is the display process state; `shutting` the clean-shutdown signal; `wphase` the worker's program counter.

Modelling the store read/write lock as *one* mutually-exclusive slot (`readLock` for the sole reader, `storeLockMut` for the one writer) is sound because there is exactly one reader thread in this design — the worker. A read/write lock exists to allow concurrent readers; with a single reader the only exclusion that can ever fire is reader-versus-writer, which a single slot captures.

`menuShown` is modelled at the level of the worker's delivery, not at the level of the display process's live menu content, and the two differ in one honest way. For scenes, the model resets `displayed` to empty on a respawn (`Ensure`) and proves the content property I4 — so it must track what the fresh display shows. For menus it proves the weaker delivery property I6 — "a marked change is eventually pushed or stays marked" — so it tracks whether a push is owed (`menusDirty`) and whether the last owed push landed (`menuShown`). The re-establishment of a menu on a freshly respawned display over the client handshake is a separate subsystem, outside this replicator's dirty-signal discipline, exactly as the client-side replay of scenes is outside it; the model therefore leaves `menuShown` unchanged across `Ensure` and `Reconn` and holds the worker to its own obligation: never drop a marked menu change.

5 Initialization

Init

$$Repl'$$

$$store' = (\lambda\, s : SCENE \bullet vempty)$$
$$displayed' = (\lambda\, s : SCENE \bullet vempty)$$
$$dirty' = \varnothing$$
$$batch' = \varnothing$$
$$drained' = \varnothing$$
$$cleared' = clear$$
$$batchCleared' = clear$$
$$menusDirty' = clear$$
$$batchMenus' = clear$$
$$menuShown' = set$$
$$snapScene' = \varnothing$$
$$snapVal' = vempty$$
$$storeLockMut' = \varnothing$$
$$mutScene' = \varnothing$$
$$readLock' = free$$
$$clientLock' = free$$
$$display' = dup$$
$$shutting' = clear$$
$$wphase' = widle$$

A single initial state: empty store, empty display, nothing dirty, no menu change owed and the menu delivered, no locks held, the worker idle, the display up.

6 Mutator Operations

A mutator changes the store under the write lock in two steps, so the scene passes through a torn intermediate the worker must never read. The first step takes the lock and begins the replace; the second commits the new value, marks the scene dirty, and releases the lock. Marking dirty is part of the commit: the tool calls `mark_dirty` on the same thread, immediately after `replace_scene` returns and before it yields, so no other thread observes the store between the store write and the mark.

The write lock is taken only when it is free *and* the worker is not mid-snapshot (`readLock = free`). That guard is the store lock's reader-writer exclusion; removing it from the worker's snapshot yields the buggy variant of Section 12.

ReplaceBegin

$\Delta Repl$
$m? : MUTATOR$
$s? : SCENE$

$storeLockMut = \varnothing$
$readLock = free$
$storeLockMut' = \{m?\}$
$mutScene' = \{s?\}$
$store' = store \oplus \{s? \mapsto vtorn\}$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$
$wphase' = wphase$

ReplaceCommit

$\Delta Repl$
$m? : MUTATOR$

$m? \in storeLockMut$
$store' = store \oplus (mutScene \times \{vlive\})$
$dirty' = dirty \cup mutScene$
$storeLockMut' = \varnothing$
$mutScene' = \varnothing$
$displayed' = displayed$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$
$wphase' = wphase$

A single scene can also be emptied without a clear — an `update` that strips a scene to no roots, or a session whose disconnect blanks the scenes it alone held. `EmptyScene` sets one scene's store value to `vempty` and marks it dirty, under the same write-lock discipline as a replace. It is what makes a departed scene's blank observable to the worker.

EmptyScene

$\Delta Repl$
$s? : SCENE$

$storeLockMut = \varnothing$
$readLock = free$
$store' = store \oplus \{s? \mapsto vempty\}$
$dirty' = dirty \cup \{s?\}$
$displayed' = displayed$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$
$wphase' = wphase$

The `clear` tool empties the whole store under the write lock and sets the cleared flag instead of marking a scene dirty. Modelled as one step, since the torn-read race is already exhibited by `replace`:

ClearStore

$\Delta Repl$

$storeLockMut = \varnothing$
$readLock = free$
$store' = (\lambda\, s : SCENE \bullet vempty)$
$cleared' = set$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$
$wphase' = wphase$

The `set_menu` / `set_registered_items` tools change the Hub's menu registry and flag the change, exactly as `clear` sets the cleared flag. `MarkMenus` sets the live menu flag and records that the display is now behind the registry (`menuShown' = clear`). Like the real `mark_menus`, it is queue-only: it touches no store, takes no lock, and is guarded only by `menusDirty = clear` — coalescing, since a second change while one is already pending is a no-op. It never mentions `clientLock`, `display`, or `wphase`, so no state of the display can disable it; the menu-set path is as free of display I/O as the store mutators are (I2).

MarkMenus

$\Delta Repl$

$menusDirty = clear$
$menusDirty' = set$
$menuShown' = clear$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$batchMenus' = batchMenus$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$
$wphase' = wphase$

7 The Worker Loop

The worker waits on the condition, wakes when something is dirty, the screen was cleared, or the menu registry changed, drains the whole set, and pushes each scene by copying its current roots under the read lock and sending the copy outside the lock. Coalescing needs no extra mechanism: many marks of one scene add it to the set once, and the worker reads that scene's current store value when it snapshots, so the display receives the newest value, never a half-finished one.

Wake

$\Delta Repl$

$wphase = widle$
$shutting = clear$
$(dirty \neq \varnothing \lor cleared = set \lor menusDirty = set)$
$wphase' = wdrain$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$

`Drain` takes the whole changed-scene set at once, snapshots the cleared flag into `batchCleared` and the menu flag into `batchMenus` for the duration of the cycle, and resets both live flags — exactly as `wait_and_drain` builds an immutable `DrainedBatch` and clears the signal. A clear or a menu change that arrives after this point re-sets its live flag for the next cycle.

Drain

$\Delta Repl$

$wphase = wdrain$
$batch' = dirty$
$drained' = dirty$
$dirty' = \varnothing$
$batchCleared' = cleared$
$cleared' = clear$
$batchMenus' = menusDirty$
$menusDirty' = clear$
$menuShown' = menuShown$
$wphase' = wblank$
$store' = store$
$displayed' = displayed$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$

At `wblank` the worker blanks the display first if the batch carried a clear, and only then turns to the menu and the scenes. `ApplyClear` pushes the blank; `SkipBlank` is the no-clear case. Both advance to `wmenu`. `batchCleared` is *not* consumed here: it survives the blank so that the per-scene empty skip and any recovery re-mark can still see that this was a clearing batch.

ApplyClear

$\Delta Repl$

$wphase = wblank$
$batchCleared = set$
$displayed' = (\lambda\, s : SCENE \bullet vempty)$
$wphase' = wmenu$
$store' = store$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$

SkipBlank

$\Delta Repl$

$wphase = wblank$
$batchCleared = clear$
$wphase' = wmenu$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$

At `wmenu` the worker pushes the menu registry if the batch carried a menu change, and otherwise skips straight to the scenes. The push is one send, so it takes the client lock and goes to `wmsend`; the store read lock is not held, so I3 is trivially preserved on this leg. Reading the registry is instantaneous and does not appear in the state, because the menu is payload-less: whatever the registry holds when `MenuSendOk` fires is what the display receives, which is why a stale re-mark can never lose a newer change.

SendMenusBegin

$\Delta Repl$

$wphase = wmenu$
$batchMenus = set$
$clientLock' = held$
$wphase' = wmsend$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$display' = display$
$shutting' = shutting$

MenuSkip

$\Delta Repl$

$wphase = wmenu$
$batchMenus = clear$
$wphase' = wproc$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$

The menu send has the same three outcomes as a scene send, keyed on the display process. On success the display now shows the current registry, so the delivery obligation is met (`menuShown' = set`) and the worker turns to the scenes. A wedged display sends the worker to the reap path; a dead peer to the reconnect path. In both failures `menuShown` stays `clear`, so the obligation is still owed until a recovery re-marks it and a later cycle re-delivers.

MenuSendOk

$\Delta Repl$

$wphase = wmsend$
$display = dup$
$clientLock' = free$
$menuShown' = set$
$wphase' = wproc$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$display' = display$
$shutting' = shutting$

MenuSendStuck

$\Delta Repl$

$wphase = wmsend$
$display = dstuck$
$clientLock' = free$
$wphase' = wreap$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$display' = display$
$shutting' = shutting$

MenuSendDead

$\Delta Repl$

$wphase = wmsend$
$display = ddead$
$clientLock' = free$
$wphase' = wreconn$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$display' = display$
$shutting' = shutting$

`Snap` takes the store read lock, picks one scene still in the batch, and copies its current value. The read lock is available only when no mutator holds the write lock (`storeLockMut` = $\varnothing$); that guard is what stops the worker reading a torn scene. Blank-before-repaint is now enforced by the `wblank` phase, which precedes `wproc`, so `Snap` carries no cleared guard of its own.

Snap

$\Delta Repl$

$wphase = wproc$
$batch \neq \varnothing$
$storeLockMut = \varnothing$
$(\exists\, s : SCENE \bullet$
$\quad s \in batch \land snapScene' = \{s\} \land snapVal' = store\, s)$
$readLock' = held$
$wphase' = wsnap$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$

From `wsnap` the worker either sends the copy or, when the copy is empty and the batch already blanked the whole display with a clear, skips it. `SendBegin` releases the store read lock and only then takes the client send lock; this release-before-acquire is the whole of I3. Its guard `¬ (snapVal = vempty ∧ batchCleared = set)` is the send/skip partition: an empty scene under a clear is the one case that does not send.

SendBegin

$\Delta Repl$

$wphase = wsnap$
$\neg\ (snapVal = vempty \land batchCleared = set)$
$readLock' = free$
$clientLock' = held$
$wphase' = wsend$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$display' = display$
$shutting' = shutting$

`SendSkip` is the complementary case: the scene is empty and the clear already blanked the whole display, so the worker copied nothing worth sending. It releases the read lock, drops the scene from the

batch, and sends nothing — `displayed` is untouched, since the clear's blank already set that scene to `vempty`.

SendSkip

$\Delta Repl$

$wphase = wsnap$
$snapVal = vempty \land batchCleared = set$
$readLock' = free$
$batch' = batch \setminus snapScene$
$drained' = drained$
$snapScene' = \varnothing$
$snapVal' = vempty$
$wphase' = wproc$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$

The time-limited send has three outcomes, keyed on the display process. On success the display now shows the copied value and the scene leaves the batch. When the copied value is `vempty` — an emptied or departed scene sent without a clear — this is the blank that makes it vanish from its own frame.

SendOk

$\Delta Repl$

$wphase = wsend$
$display = dup$
$clientLock' = free$
$displayed' = displayed \oplus (snapScene \times \{snapVal\})$
$batch' = batch \setminus snapScene$
$drained' = drained$
$snapScene' = \varnothing$
$snapVal' = vempty$
$wphase' = wproc$
$store' = store$
$dirty' = dirty$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$display' = display$
$shutting' = shutting$

On a send timeout (`BlockingIOError`) the display is alive but wedged. The send gives up within its time limit and releases the client lock, and the worker turns to the reap-respawn path. The batch is kept: the respawn re-marks every live scene, which re-covers the scene in flight.

SendStuck

$\Delta Repl$

$wphase = wsend$
$display = dstuck$
$clientLock' = free$
$wphase' = wreap$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$display' = display$
$shutting' = shutting$

On a dead peer (`OSError`) there is nothing to kill; the worker closes the dead descriptor and reconnects.

SendDead

$\Delta Repl$

$wphase = wsend$
$display = ddead$
$clientLock' = free$
$wphase' = wreconn$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$display' = display$
$shutting' = shutting$

8 Reap, Respawn, Reconnect

A stuck display still answers new connections, so it must be killed before a fresh one is started. `Reap` terminates the wedged process; `Ensure` starts a fresh, empty one; `Remark` re-marks the work so the display is repainted from the store's current state, re-marks the clear if the failed batch carried one,

and re-marks the menu flag if the failed batch carried one. `Ensure` is enabled only from `ddead`: a fresh display is never spawned while one is alive, which is I5.

The re-mark set is `drained` ∪ the live scenes: every scene the failed batch drained, plus every scene the store still holds live. The two unions cover complementary gaps. The live scenes must be re-marked because a fresh or reconnected display shows none of them until they are pushed again. The *drained* scenes must be re-marked because a batch can drain a scene that is *not* live — an emptied or departed scene whose only remaining job is to be blanked; that blank can be lost to the failed send, and it is absent from the live set, so re-marking only the live scenes would drop it. Unioning `drained` keeps the emptied scene's blank in the queue until it lands. `SendRecovery._remark` does exactly this: it re-marks `batch.scenes` together with `live_scene_ids()`.

The clear re-mark is the third part of the heal. A cleared scene is empty in the store, so it is absent from the live set, and `clear` marks no scene dirty, so it is absent from `drained` as well; without re-marking the clear, a reconnect to the same surviving display would leave the old, pre-clear scene on screen forever. Re-marking the clear re-arms the live flag, so the next cycle blanks the display again before repainting; blank-then-repaint is idempotent. The re-mark is expressed against `batchCleared` — the clear this failed batch carried — and merges into whatever the live flag already holds, so a fresh clear that arrived mid-cycle is never lost.

The menu re-mark is the fourth part, and it is the guarantor of I6. A menu send that failed left `menuShown = clear`, and the drained menu flag is in `batchMenus`; re-marking it against `batchMenus` re-arms the live `menusDirty` so the next cycle re-reads the registry fresh and re-delivers. `SendRecovery._requeue` does exactly this: it calls `mark_menus` when the batch's `menus_dirty` was set. Dropping this re-mark is the fidelity variant of Section 12, under which a menu change whose send failed is neither delivered nor re-marked — an I6 violation.

Reap

$\Delta Repl$

$wphase = wreap$
$display' = ddead$
$wphase' = wensure$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$shutting' = shutting$

Ensure

$\Delta Repl$

$wphase = wensure$
$display = ddead$
$display' = dup$
$displayed' = (\lambda\, s : SCENE \bullet vempty)$
$snapScene' = \varnothing$
$snapVal' = vempty$
$wphase' = wremark$
$store' = store$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$shutting' = shutting$

Remark

$\Delta Repl$

$wphase = wremark$
$dirty' = (dirty \cup drained) \cup \{s : SCENE \mid store\, s = vlive\}$
$(batchCleared = set \Rightarrow cleared' = set)$
$(batchCleared = clear \Rightarrow cleared' = cleared)$
$(batchMenus = set \Rightarrow menusDirty' = set)$
$(batchMenus = clear \Rightarrow menusDirty' = menusDirty)$
$batch' = \varnothing$
$drained' = \varnothing$
$wphase' = wproc$
$store' = store$
$displayed' = displayed$
$batchCleared' = batchCleared$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$

`Reconn` handles the dead-peer path, and it is modelled adversarially. A plain `OSError` does not mean the display *process* died; it means *this connection* did. The display is a long-lived process that keeps its last-received scene copy and renders it every frame, so a reconnect can land on the same still-running display, showing exactly what it showed before the drop. `Reconn` therefore leaves `displayed` unchanged — the worst case, stale content and all — rather than pretending a fresh blank display. There is no process to kill, so it reconnects and re-marks in one step: `drained` ∪ the live scenes, plus the clear and the menu flag the batch carried. I4 holds *because of* that re-mark, not because of any reset: every scene

the failed cycle touched, and every live scene, is re-queued, so the stale display cannot stay wrong once the queue drains.

Modelling the reconnect this way is what makes the `drained` half of the re-mark load-bearing. Under a reset-to-empty reconnect a fresh display would already blank every emptied scene, and re-marking `drained` would change nothing an observer could see. Against a stale-content display it is the only thing that re-blanks an emptied scene whose blank was lost to the failed send; Section 12 removes it and reproduces the linger.

Reconn

$\Delta Repl$

$wphase = wreconn$
$display' = dup$
$displayed' = displayed$
$dirty' = (dirty \cup drained) \cup \{s : SCENE \mid store\ s = vlive\}$
$(batchCleared = set \Rightarrow cleared' = set)$
$(batchCleared = clear \Rightarrow cleared' = cleared)$
$(batchMenus = set \Rightarrow menusDirty' = set)$
$(batchMenus = clear \Rightarrow menusDirty' = menusDirty)$
$batch' = \varnothing$
$drained' = \varnothing$
$snapScene' = \varnothing$
$snapVal' = vempty$
$wphase' = wproc$
$store' = store$
$batchCleared' = batchCleared$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$shutting' = shutting$

`CycleFail` is the generic failure outcome: any error inside the cycle other than the two socket outcomes — a display that will not spawn, a reconnect that is refused, or any exception the two socket handlers do not catch. It can strike while the worker is sending a scene (`wsend`) or the menu (`wmsend`) or mid-heal (`wreap`, `wensure`, `wreconn`). The drained work must not be lost, so the worker re-marks it — `drained`, plus the clear and the menu flag the batch carried — releases whatever locks it held, and returns to `widle` to back off and retry. Restoring `drained` rather than the unsent remainder `batch` is what saves an already-sent scene: if the batch carried a clear, the re-marked clear will blank the whole display next cycle, so every scene the batch drained — including those already sent — must be repainted, or a painted scene would be blanked and never redrawn.

Unlike `Remark` and `Reconn`, `CycleFail` does *not* union the live scenes: it re-marks `drained` alone. This matches `SendRecovery.restore`, which re-queues `batch.scenes` without `live_scene_ids`, and it is sound because `CycleFail` does not replace the display. A live scene the failed cycle did not touch is still shown correctly on the same unblanked display, so it needs no repaint; only the interrupted cycle's drained work does. The live union is needed only when the display is reset (`Remark`) or reconnected onto stale content (`Reconn`), where a live scene may not be shown.

CycleFail

$\Delta Repl$

$wphase \in \{wsend, wmsend, wreap, wensure, wreconn\}$
$dirty' = dirty \cup drained$
$(batchCleared = set \Rightarrow cleared' = set)$
$(batchCleared = clear \Rightarrow cleared' = cleared)$
$(batchMenus = set \Rightarrow menusDirty' = set)$
$(batchMenus = clear \Rightarrow menusDirty' = menusDirty)$
$batch' = \varnothing$
$drained' = \varnothing$
$batchCleared' = clear$
$batchMenus' = clear$
$menuShown' = menuShown$
$snapScene' = \varnothing$
$snapVal' = vempty$
$readLock' = free$
$clientLock' = free$
$wphase' = widle$
$store' = store$
$displayed' = displayed$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$display' = display$
$shutting' = shutting$

9 Ending a Cycle, Clear, Shutdown

A cleared cycle blanks the display *before* it repaints the batch: the `wblank` phase runs `ApplyClear` first, and the scenes are reached only at `wproc`, so no scene is painted until the clear has been applied. The worker then repaints each batch scene from the store's current value, so a `show` that coalesced after the `clear` survives.

This is a deliberate departure from the original design text, which pushed the clear *after* the batch (`for scene in batch: push(...); if was_cleared: push_clear()`). Model-checking I4 against that order found a lost update: a `clear` followed by a `show` within the worker's 16 ms coalescing window painted the new scene and then blanked it (Section 11). The clear-first order is convergent for both `show`-then-`clear` and `clear`-then-`show`, and the shipped `_attempt` implements it: `clear_async` first, then the menu push, then the per-scene repaint. The clear's own send is modelled as an immediate blank; its send has the same time limit and the same failure outcomes as a scene send, which the send operations already cover.

When the batch is empty — every scene sent or skipped — the worker drops the batch's carried clear and menu flag and goes idle.

ProcDone

$\Delta Repl$

$wphase = wproc$
$batch = \varnothing$
$batchCleared' = clear$
$batchMenus' = clear$
$menuShown' = menuShown$
$wphase' = widle$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = \varnothing$
$cleared' = cleared$
$menusDirty' = menusDirty$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$

The display fails independently of the worker: an environment step may wedge it or kill it. These are the events the worker's next send detects.

DisplayWedge

$\Delta Repl$

$display = dup$
$display' = dstuck$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$shutting' = shutting$
$wphase' = wphase$

DisplayCrash

$\Delta Repl$

$display = dup$
$display' = ddead$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$shutting' = shutting$
$wphase' = wphase$

At clean shutdown the worker makes one last bounded flush of whatever is still dirty and then stops. The flush is best-effort: losing the last frame is acceptable because the process is exiting. `Restart` re-enters a fresh idle state, modelling luxd starting again, and keeps the model live so the deadlock check reports only genuine lock deadlocks, never this deliberate terminal stop.

ShutdownReq

$\Delta Repl$

$shutting = clear$
$shutting' = set$
$store' = store$
$displayed' = displayed$
$dirty' = dirty$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$wphase' = wphase$

Flush

$\Delta Repl$

$wphase = widle$
$shutting = set$
$dirty' = \varnothing$
$wphase' = wstopped$
$store' = store$
$displayed' = displayed$
$batch' = batch$
$drained' = drained$
$cleared' = cleared$
$batchCleared' = batchCleared$
$menusDirty' = menusDirty$
$batchMenus' = batchMenus$
$menuShown' = menuShown$
$snapScene' = snapScene$
$snapVal' = snapVal$
$storeLockMut' = storeLockMut$
$mutScene' = mutScene$
$readLock' = readLock$
$clientLock' = clientLock$
$display' = display$
$shutting' = shutting$

Restart

$\Delta Repl$

$wphase = wstopped$
$store' = (\lambda\, s : SCENE \bullet vempty)$
$displayed' = (\lambda\, s : SCENE \bullet vempty)$
$dirty' = \varnothing$
$batch' = \varnothing$
$drained' = \varnothing$
$cleared' = clear$
$batchCleared' = clear$
$menusDirty' = clear$
$batchMenus' = clear$
$menuShown' = set$
$snapScene' = \varnothing$
$snapVal' = vempty$
$storeLockMut' = \varnothing$
$mutScene' = \varnothing$
$readLock' = free$
$clientLock' = free$
$display' = dup$
$shutting' = clear$
$wphase' = widle$

10 Invariants

Six properties must hold across all reachable states. None is placed in the `Repl` schema: a predicate placed there would be enforced as a guard on every successor, silently disabling any operation that would break it and thereby masking the very defect this model exists to catch.

Invariant 1 — single writer. Only the worker ever holds the client send lock, and it holds it only while sending — a scene at `wsend` or the menu at `wmsend`:

$$clientLock = held \Rightarrow wphase \in \{wsend, wmsend\}.$$

No mutator operation mentions `clientLock` in its effect, so no mutator can ever hold it. The design removes the two second-writer paths the current code has — the inline resend in the click dispatch and the beads app's direct render — so that the worker's `wsend` and `wmsend` are the only places a send happens.

Invariant 2 — agent-path liveness. No mutator operation is guarded by `clientLock`, `display`, or `wphase`: `ReplaceBegin`, `ReplaceCommit`, `EmptyScene`, `ClearStore`, and `MarkMenus` depend only on the store write lock being free (and `MarkMenus` only on `menusDirty`). So no state of the display or of the worker's send can disable a mutator. A mutator's begin does also require `readLock = free`, the ordinary reader-writer exclusion of the store lock; but the worker holds that read lock only across `Snap`'s in-memory copy and releases it in `SendBegin` before any send, so a mutator waits at most for one snapshot copy and never for display I/O. The witness (Section 11) is a reachable state in which the worker is stuck sending to a wedged display while a mutator holds the write lock and is free to commit.

Invariant 3 — no lock cycle. No thread holds the store lock and the client send lock at once:

$$\neg\ (readLock = held \wedge clientLock = held).$$

The worker takes the read lock in `Snap`, releases it in `SendBegin` *before* taking the client lock, and no mutator ever takes the client lock. The menu send takes the client lock at `wmsend` while holding no read lock, so it cannot form a cycle either. The two locks are therefore always acquired in one order and never held together, so no cyclic wait can form.

Invariant 4 — no lost update. Whenever the system is at rest with the display up, the display shows the store's latest value for every scene. "At rest" means the worker is idle, nothing is dirty, cleared, or menu-marked, no mutator holds the write lock, and no shutdown is in progress:

$$Quiescent \Rightarrow (\forall\, s : SCENE \bullet displayed\ s = store\ s),$$

where

$$\begin{aligned} Quiescent \ \widehat{=}\ & wphase = widle \wedge dirty = \varnothing \wedge cleared = clear \\ & \wedge\ menusDirty = clear \wedge storeLockMut = \varnothing \wedge shutting = clear \wedge display = dup. \end{aligned}$$

Every recovery re-marks at least `drained` plus the carried clear, so the system cannot come to rest until that work has been pushed, which restores `displayed = store`. The two display-replacing recoveries also union the live scenes: `Remark` follows a respawn onto a fresh, empty display, and `Reconn` follows a reconnect onto a stale-content display, and in both a live scene may not be shown, so re-marking it is what closes that gap. `CycleFail` does not replace the display, so a live scene it did not touch is still shown correctly and needs no repaint; it re-marks `drained` alone. In every case the re-mark covers the two gaps that can actually open — a live scene a replaced display is not showing, and an emptied scene whose blank was lost — so no scene can stay wrong at rest. An emptied scene reaches rest only once its blank has been pushed (`SendOk` with `snapVal = vempty`), or, under a clear, once the whole-display blank has set it empty; either way `displayed s = store s = vempty`. `batchCleared` does not appear in `Quiescent` because it is `clear` at every `widle` state: every path into `widle` — `ProcDone`, `CycleFail`, `Init`, `Restart` — resets it.

Invariant 5 — one display. A fresh display is spawned only after the old one is killed: `Ensure` is enabled only from `display = ddead`, and `Reap` is the only step that reaches `ddead` on the recovery path. So the worker never holds two live displays.

Invariant 6 — menu-delivery durability. Whenever the system is at rest with the display up, no marked menu change is left undelivered:

$$Quiescent \Rightarrow menuShown = set.$$

`menuShown` goes `clear` only at `MarkMenus`, which also sets `menusDirty`; it goes `set` only at `MenuSendOk`, a successful push. A `Quiescent` state requires `menusDirty = clear`, and the only way `menusDirty` returns to `clear` after a mark is `Drain`, which carries the change into `batchMenus`. That drained change is then either delivered — `SendMenusBegin` then `MenuSendOk`, which sets `menuShown = set` — or its send fails, and every failure path re-marks `menusDirty` against `batchMenus` (`Remark`, `Reconn`, `CycleFail`), which re-arms the live flag and so denies `Quiescent` until a later cycle re-delivers. Hence a rest state with the display up cannot be reached with `menuShown = clear`: the marked change was pushed, or it is still marked and the system is not at rest. This is the delivery obligation of the replicator alone; that a freshly respawned display also re-obtains its registered items over the client handshake is a separate mechanism the model does not rely on.

11 Verification

Type-checking is a `fuzz` pass:

```
fuzz docs/hub_replicator.tex
```

I3, I4, I5, and I6 are state properties, checked by asking ProB whether their negation is *reachable*. A goal reported *not found* means the invariant holds on every reachable state:

```
# Invariant 3 (must report: goal NOT found)
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "readLock = held & clientLock = held" \
  -p DEFAULT_SETSIZE 2

# Invariant 4 (must report: goal NOT found)
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = widle & dirty = {} & cleared = clear &
         menusDirty = clear & storeLockMut = {} & shutting = clear &
         display = dup &
         card({s | s : SCENE & displayed(s) /= store(s)}) > 0" \
  -p DEFAULT_SETSIZE 2

# Invariant 5 (must report: goal NOT found)
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = wensure & display /= ddead" \
  -p DEFAULT_SETSIZE 2

# Invariant 6 (must report: goal NOT found)
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = widle & dirty = {} & cleared = clear &
         menusDirty = clear & storeLockMut = {} & shutting = clear &
         display = dup & menuShown = clear" \
  -p DEFAULT_SETSIZE 2
```

I1 is corroborated by a reachability goal that must be *not found* — no state holds the client lock outside a scene or menu send:

```
# Invariant 1 (must report: goal NOT found)
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "clientLock = held & wphase /= wsend & wphase /= wmsend" \
  -p DEFAULT_SETSIZE 2
```

I2 is a liveness witness that must be *found*: a reachable state in which the worker is sending to a wedged display while a mutator holds the write lock and can still commit — the agent path is not blocked by display I/O:

```
# Invariant 2 (must report: goal FOUND)
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "display = dstuck & clientLock = held & storeLockMut /= {}" \
  -p DEFAULT_SETSIZE 2
```

Deadlock freedom is the full-state-space check. `Restart` keeps the model live, so any deadlock reported would be a genuine lock deadlock:

```
# Deadlock freedom (must report: no deadlock / no counterexample)
probcli docs/hub_replicator.tex -model_check -p DEFAULT_SETSIZE 2
```

The recovery-and-blanking behaviours and the menu leg must be reachable, or their invariants would hold only because the machinery is dead. Each of these witnesses must be *found*:

```
# Generic cycle failure is reachable and restores its batch
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = wsend & batch /= {}" -p DEFAULT_SETSIZE 2

# A batch that carried a clear reaches recovery (clear re-mark is live)
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = wremark & batchCleared = set" -p DEFAULT_SETSIZE 2

# The empty-scene-under-clear skip is reachable
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = wsnap & snapVal = vempty & batchCleared = set" \
  -p DEFAULT_SETSIZE 2

# An emptied scene is blanked without a clear (behaviour 3, happy path)
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = wsend & snapVal = vempty & batchCleared = clear" \
  -p DEFAULT_SETSIZE 2

# The menu send is reachable (the menu leg is live)
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = wmsend & batchMenus = set" -p DEFAULT_SETSIZE 2

# A menu-carrying batch reaches recovery (the menu re-mark is live)
probcli docs/hub_replicator.tex -model_check -nodead \
  -goal "wphase = wremark & batchMenus = set" -p DEFAULT_SETSIZE 2
```

Finding — clear ordering. Model-checking I4 against the original design document's stated order — push the batch scenes, then `if was_cleared: push_clear()` — returns *goal found*. Replace the `wblank`-phase blank (which happens first) with a `PushClear` that blanks at the end of the cycle, and this trace is reachable:

```
ClearStore              ; store emptied, cleared set
ReplaceBegin/Commit s1  ; a show() lands after the clear, within
                        ;   the 16ms coalescing window; store s1 = vlive
Drain                   ; batch = {s1}, batchCleared = set
Snap; SendBegin; SendOk ; display now shows s1 = vlive
PushClear               ; display blanked to empty
-- rest: displayed s1 = vempty, store s1 = vlive  (lost update)
```

A `clear` immediately followed by a `show` is an ordinary sequence, so this is a reachable defect, not a corner case. The clear-first order — the `wblank` phase preceding `wproc` — closes it: with clear-first the goal returns *not found*, and the shipped `_attempt` sends `clear_async` before the per-scene repaint.

Note — the two reconnect models. The reap path and the dead-peer path differ in one honest way, and the model keeps them apart. `Reap`/`Ensure` kill the wedged process and spawn a new one, which genuinely starts blank, so the model resets `displayed` to empty. `Reconn` does not spawn anything: an

`OSError` kills the connection, not necessarily the display process, and that process keeps its last-received copy and renders it every frame. The model therefore leaves `displayed` unchanged on `Reconn` — the adversarial, faithful case of a reconnect onto a still-running display that still shows the pre-failure scene.

That choice is what makes the `drained` half of the re-mark load-bearing. Had `Reconn` reset to empty, a fresh display would already blank every emptied scene, and unioning `drained` into the re-mark would change nothing an observer could see — the model would verify clean while being too abstract to guard the fix. Against a stale-content reconnect it is exactly the union that saves an emptied scene: its blank, lost to the failed send, is absent from the live set but present in `drained`, so re-marking `drained` ∪ live re-queues it. Section 12 removes that union and reproduces the linger the fix closes. This is the gap flagged in the previous revision of this report, now closed in both the code (`SendRecovery._remark` unions `batch.scenes`) and the model.

12 Fidelity: reproducing the known defects

A model that cannot reproduce the known defects proves nothing. This specification guards three, and removing any one fix makes the corresponding defect reachable.

The store race. The first buggy variant removes the store's reader-writer exclusion from the worker's snapshot. Concretely, delete the guard `storeLockMut = ∅` from `Snap`, so the worker can copy a scene's value while a mutator is inside `replace_scene` and the scene is `vtorn`. The torn read then becomes reachable:

1. Mutator m runs `ReplaceBegin` on scene s: it takes the write lock and sets `store` s `= vtorn`, with `storeLockMut = {m}`.
2. Meanwhile the worker has a batch containing s and is at `wproc`. Without the exclusion guard, `Snap` fires: it copies `snapVal = store` s `= vtorn`.
3. The worker is now about to send a half-updated scene: a state with `snapVal = vtorn`.

The property that separates the two designs is therefore

```
# must be NOT found for the correct spec, FOUND for the buggy variant
probcli <spec>.tex -model_check -nodead \
  -goal "snapVal = vtorn" -p DEFAULT_SETSIZE 2
```

Under the intact exclusion the worker's `Snap` cannot fire while any mutator holds the write lock, so `snapVal` is never `vtorn`; the reachability search returns *goal not found*. With the guard removed it returns *goal found* — the trace above. That difference is the evidence that the model detects the race the store lock was added to prevent.

The emptied-scene linger. The second buggy variant removes the `drained` half of the recovery re-mark. Concretely, in `Reconn` replace `dirty′ = (dirty ∪ drained) ∪ {s | store s = vlive}` with the live-only `dirty′ = dirty ∪ {s | store s = vlive}` — the pre-fix recovery that re-marked only the live scenes. With `Reconn` modelled against a stale-content display, an emptied scene's lost blank is then never re-queued:

1. `show` paints scene s: `store` s `= vlive`, `displayed` s `= vlive`.
2. `EmptyScene` s: `store` s `= vempty`, s marked dirty. Its blank is now the worker's job.
3. The worker drains s and sends its blank; the send raises `OSError`. `SendDead` → `wreconn`.
4. `Reconn` leaves `displayed` s `= vlive` (stale display) and, live-only, re-marks nothing for s — s is empty in the store, so it is not live. s is dropped.
5. At rest: `displayed` s `= vlive`, `store` s `= vempty` — a `Quiescent` lost update.

The property that separates the two designs is I4 itself:

```
# must be NOT found for the correct spec, FOUND for the live-only variant
probcli <spec>.tex -model_check -nodead \
  -goal "wphase = widle & dirty = {} & cleared = clear &
         menusDirty = clear & storeLockMut = {} & shutting = clear &
         display = dup &
         card({s | s : SCENE & displayed(s) /= store(s)}) > 0" \
  -p DEFAULT_SETSIZE 2
```

With the `drained` union intact the emptied scene is re-queued and reblanked, so the goal returns *not found*. With it removed the goal returns *found* — the trace above. That difference is the evidence that the model detects the defect reviewers found in the live-only recovery, which `SendRecovery._remark`'s union of `batch.scenes` closes.

The dropped menu change. The third buggy variant removes the menu re-mark from the recovery. Concretely, drop the two lines `(batchMenus = set ⇒ menusDirty′ = set)` and `(batchMenus = clear ⇒ menusDirty′ = menusDirty)` from `Remark`, `Reconn`, and `CycleFail`, replacing them with an unconditional `menusDirty′ = menusDirty` — the recovery that re-queues scenes and the clear but forgets the menu. A menu change whose send failed is then neither delivered nor re-marked:

1. `MarkMenus`: `menusDirty = set`, `menuShown = clear` — a menu change is owed.
2. `Drain`: `batchMenus = set`, `menusDirty = clear`.
3. `SkipBlank`; `SendMenusBegin`: the worker takes the client lock to push the menu, reaching `wmsend`.
4. The display is wedged (`DisplayWedge` beforehand), so `MenuSendStuck` fires: `wreap`, `menuShown` still `clear`.
5. `Reap`; `Ensure`; `Remark` — but, live-only, it does not re-mark `menusDirty`. `batchMenus` is consumed.
6. No scenes remain, so `ProcDone` returns the worker to `widle`.
7. At rest: `menusDirty = clear`, `menuShown = clear` — a `Quiescent` state with a menu change that was marked, never delivered, and no longer marked. I6 is violated.

The property that separates the two designs is I6 itself:

```
# must be NOT found for the correct spec, FOUND for the no-remark variant
probcli <spec>.tex -model_check -nodead \
  -goal "wphase = widle & dirty = {} & cleared = clear &
         menusDirty = clear & storeLockMut = {} & shutting = clear &
         display = dup & menuShown = clear" \
  -p DEFAULT_SETSIZE 2
```

With the menu re-mark intact, a failed menu send re-arms `menusDirty`, so the system cannot rest until a later cycle re-reads the registry and re-delivers; the goal returns *not found*. With it removed the goal returns *found* — the trace above. That difference is the evidence that the model detects a dropped menu change, which `SendRecovery._requeue`'s call to `mark_menus` closes.